CHAPTER 3
Modularity

First, we want to untangle some common terms used and overused in discussions about architecture surrounding modularity and provide definitions for use throughout the book.

> 95% of the words [about software architecture] are spent extolling the benefits of "modularity" and that little, if anything, is said about how to achieve it.
>
> —Glenford J. Myers (1978)

Different platforms offer different reuse mechanisms for code, but all support some way of grouping related code together into *modules*. While this concept is universal in software architecture, it has proven slippery to define. A casual internet search yields dozens of definitions, with no consistency (and some contradictions). As you can see from the quote from Myers, this isn't a new problem. However, because no recognized definition exists, we must jump into the fray and provide our own definitions for the sake of consistency throughout the book.

Understanding modularity and its many incarnations in the development platform of choice is critical for architects. Many of the tools we have to analyze architecture (such as metrics, fitness functions, and visualizations) rely on these modularity concepts. Modularity is an organizing principle. If an architect designs a system without paying attention to how the pieces wire together, they end up creating a system that presents myriad difficulties. To use a physics analogy, software systems model complex systems, which tend toward entropy (or disorder). Energy must be added to a physical system to preserve order. The same is true for software systems: architects must constantly expend energy to ensure good structural soundness, which won't happen by accident.

Preserving good modularity exemplifies our definition of an *implicit* architecture characteristic: virtually no project features a requirement that asks the architect to ensure good modular distinction and communication, yet sustainable code bases require order and consistency.

Definition

The dictionary defines *module* as "each of a set of standardized parts or independent units that can be used to construct a more complex structure." We use *modularity* to describe a logical grouping of related code, which could be a group of classes in an object-oriented language or functions in a structured or functional language. Most languages provide mechanisms for modularity (`package` in Java, `namespace` in .NET, and so on). Developers typically use modules as a way to group related code together. For example, the `com.mycompany.customer` package in Java should contain things related to customers.

Languages now feature a wide variety of packaging mechanisms, making a developer's chore of choosing between them difficult. For example, in many modern languages, developers can define behavior in functions/methods, classes, or packages/namespaces, each with different visibility and scoping rules. Other languages complicate this further by adding programming constructs such as the metaobject protocol to provide developers even more extension mechanisms.

Architects must be aware of how developers package things because it has important implications in architecture. For example, if several packages are tightly coupled together, reusing one of them for related work becomes more difficult.

Modular Reuse Before Classes

Developers who predate object-oriented languages may puzzle over why so many different separation schemes commonly exist. Much of the reason has to do with backward compatibility, not of code but rather for how developers think about things. In March of 1968, Edsger Dijkstra published a letter in the *Communications of the ACM* entitled "Go To Statement Considered Harmful." He denigrated the common use of the `GOTO` statement common in programming languages at the time that allowed nonlinear leaping around within code, making reasoning and debugging difficult.

This paper helped usher in the era of *structured* programming languages, exemplified by Pascal and C, which encouraged deeper thinking about how things fit together. Developers quickly realized that most of the languages had no good way to group like things together logically. Thus, the short era of *modular* languages was born, such as Modula (Pascal creator Niklaus Wirth's next language) and Ada. These languages had the programming construct of a *module*, much as we think about packages or namespaces today (but without the classes).

The modular programming era was short-lived. Object-oriented languages became popular because they offered new ways to encapsulate and reuse code. Still, language designers realized the utility of modules, retaining them in the form of packages, namespaces, etc. Many odd compatibility features exist in languages to support these different paradigms. For example, Java supports modular (via packages and package-level initialization using static initializers), object-oriented, and functional paradigms, each programming style with its own scoping rules and quirks.

For discussions about architecture, we use modularity as a general term to denote a related grouping of code: classes, functions, or any other grouping. This doesn't imply a physical separation, merely a logical one; the difference is sometimes important. For example, lumping a large number of classes together in a monolithic application may make sense from a convenience standpoint. However, when it comes time to restructure the architecture, the coupling encouraged by loose partitioning becomes an impediment to breaking the monolith apart. Thus, it is useful to talk about modularity as a concept separate from the physical separation forced or implied by a particular platform.

It is worth noting the general concept of *namespace*, separate from the technical implementation in the .NET platform. Developers often need precise, fully qualified names for software assets to separate different software assets (components, classes, and so on) from each other. The most obvious example that people use every day is the internet: unique, global identifiers tied to IP addresses. Most languages have some modularity mechanism that doubles as a namespace to organize things: variables, functions, and/or methods. Sometimes the module structure is reflected physically. For example, Java requires that its package structure must reflect the directory structure of the physical class files.

A Language with No Name Conflicts: Java 1.0

The original designers of Java had extensive experience dealing with name conflicts and clashes in the various programming platforms at the time. The original design of Java used a clever hack to avoid the possibility of ambiguity between two classes that had the same name. For example, what if your problem domain included a catalog *order* and an installation *order*: both named *order* but with very different connotations (and classes). The solution in Java was to create the `package` namespace mechanism, along with the requirement that the physical directory structure just match the package name. Because filesystems won't allow the same named file to reside in the same directory, they leveraged the inherent features of the operating system to avoid the possibility of ambiguity. Thus, the original `classpath` in Java contained only directories, disallowing the possibility of name conflicts.

However, as the language designers discovered, forcing every project to have a fully formed directory structure was cumbersome, especially as projects became larger. Plus, building reusable assets was difficult: frameworks and libraries must be "exploded" into the directory structure. In the second major release of Java (1.2, called Java 2), designers added the `jar` mechanism, allowing an archive file to act as a directory structure on a classpath. For the next decade, Java developers struggled with getting the `classpath` exactly right, as a combination of directories and JAR files. And, of course, the original intent was broken: now two JAR files could create conflicting names on a classpath, leading to numerous war stories of debugging class loaders.

Measuring Modularity

Given the importance of modularity to architects, they need tools to understand it. Fortunately, researchers created a variety of language-agnostic metrics to help architects understand modularity. We focus on three key concepts: *cohesion*, *coupling*, and *connascence*.

Cohesion

Cohesion refers to what extent the parts of a module should be contained within the same module. In other words, it is a measure of how related the parts are to one another. Ideally, a cohesive module is one where all the parts should be packaged together, because breaking them into smaller pieces would require coupling the parts together via calls between modules to achieve useful results.

> Attempting to divide a cohesive module would only result in increased coupling and decreased readability.
>
> —Larry Constantine

Computer scientists have defined a range of cohesion measures, listed here from best to worst:

Functional cohesion
: Every part of the module is related to the other, and the module contains everything essential to function.

Sequential cohesion
: Two modules interact, where one outputs data that becomes the input for the other.

Communicational cohesion
: Two modules form a communication chain, where each operates on information and/or contributes to some output. For example, add a record to the database and generate an email based on that information.

Procedural cohesion
: Two modules must execute code in a particular order.

Temporal cohesion
: Modules are related based on timing dependencies. For example, many systems have a list of seemingly unrelated things that must be initialized at system startup; these different tasks are temporally cohesive.

Logical cohesion
: The data within modules is related logically but not functionally. For example, consider a module that converts information from text, serialized objects, or streams. Operations are related, but the functions are quite different. A common example of this type of cohesion exists in virtually every Java project in the form of the `StringUtils` package: a group of static methods that operate on `String` but are otherwise unrelated.

Coincidental cohesion
: Elements in a module are not related other than being in the same source file; this represents the most negative form of cohesion.

Despite having seven variants listed, *cohesion* is a less precise metric than *coupling*. Often, the degree of cohesiveness of a particular module is at the discretion of a particular architect. For example, consider this module definition:

`Customer Maintenance`

- `add customer`
- `update customer`
- `get customer`
- `notify customer`
- `get customer orders`
- `cancel customer orders`

Should the last two entries reside in this module or should the developer create two separate modules, such as:

`Customer Maintenance`

- `add customer`
- `update customer`
- `get customer`
- `notify customer`

`Order Maintenance`

- `get customer orders`
- `cancel customer orders`

Which is the correct structure? As always, it depends:

- Are those the only two operations for `Order Maintenance`? If so, it may make sense to collapse those operations back into `Customer Maintenance`.
- Is `Customer Maintenance` expected to grow much larger, encouraging developers to look for opportunities to extract behavior?
- Does `Order Maintenance` require so much knowledge of `Customer` information that separating the two modules would require a high degree of coupling to make it functional? This relates back to the Larry Constantine quote.

These questions represent the kind of trade-off analysis at the heart of the job of a software architect.

Surprisingly, given the subjectiveness of cohesion, computer scientists have developed a good structural metric to determine cohesion (or, more specifically, the lack of cohesion). A well-known set of metrics named the Chidamber and Kemerer Object-oriented metrics suite was developed by the eponymous authors to measure particular aspects of object-oriented software systems. The suite includes many common code metrics, such as cyclomatic complexity (see "Cyclomatic Complexity" on page 79) and several important coupling metrics discussed in "Coupling" on page 44.

The Chidamber and Kemerer Lack of Cohesion in Methods (LCOM) metric measures the structural cohesion of a module, typically a component. The initial version appears in Equation 3-1.

Equation 3-1. LCOM, version 1

$$LCOM = \begin{cases} |P| - |Q|, & \text{if } |P| > |Q| \\ 0, & \text{otherwise} \end{cases}$$

P increases by one for any method that doesn't access a particular shared field and Q decreases by one for methods that do share a particular shared field. The authors sympathize with those who don't understand this formulation. Worse, it has gradually gotten more elaborate over time. The second variation introduced in 1996 (thus the name *LCOM96B*) appears in Equation 3-2.

Equation 3-2. LCOM 96b

$$LCOM96b = \frac{1}{a}\sum_{j=1}^{a}\frac{m-\mu(Aj)}{m}$$

We wont bother untangling the variables and operators in Equation 3-2 because the following written explanation is clearer. Basically, the LCOM metric exposes incidental coupling within classes. Here's a better definition of LCOM:

LCOM
: The sum of sets of methods not shared via sharing fields

Consider a class with private fields `a` and `b`. Many of the methods only access `a`, and many other methods only access `b`. The *sum* of the sets of methods not shared via sharing fields (`a` and `b`) is high; therefore, this class reports a high LCOM score, indicating that it scores high in *lack of cohesion in methods*. Consider the three classes shown in Figure 3-1.

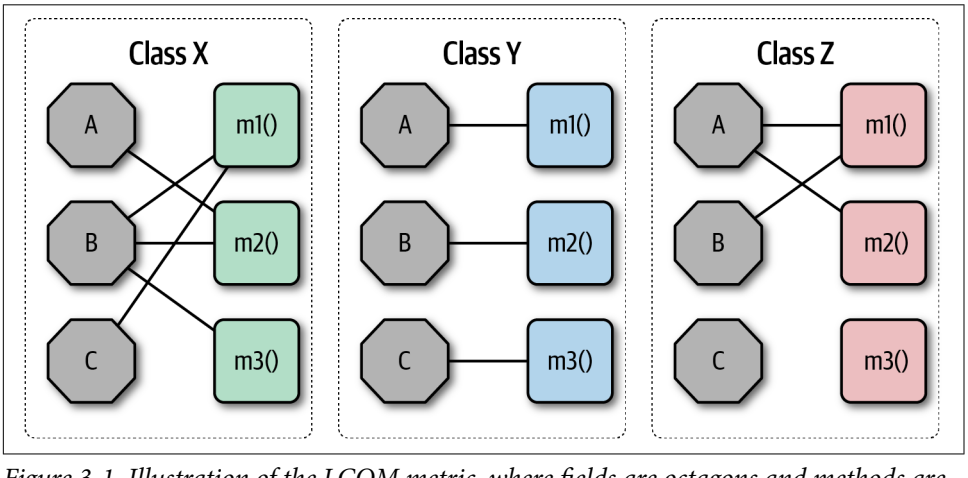

Figure 3-1. Illustration of the LCOM metric, where fields are octagons and methods are squares

In Figure 3-1, fields appear as single letters and methods appear as blocks. In `Class X`, the LCOM score is low, indicating good structural cohesion. `Class Y`, however, lacks cohesion; each of the field/method pairs in `Class Y` could appear in its own class without affecting behavior. `Class Z` shows mixed cohesion, where developers could refactor the last field/method combination into its own class.

The LCOM metric is useful to architects who are analyzing code bases in order to move from one architectural style to another. One of the common headaches when moving architectures are shared utility classes. Using the LCOM metric can help architects find classes that are incidentally coupled and should never have been a single class to begin with.

Many software metrics have serious deficiencies, and LCOM is not immune. All this metric can find is *structural* lack of cohesion; it has no way to determine logically if particular pieces fit together. This reflects back on our Second Law of Software Architecture: prefer *why* over *how*.

Coupling

Fortunately, we have better tools to analyze coupling in code bases, based in part on graph theory: because the method calls and returns form a call graph, analysis based on mathematics becomes possible. In 1979, Edward Yourdon and Larry Constantine published *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design* (Prentice-Hall), defining many core concepts, including the metrics *afferent* and *efferent* coupling. *Afferent* coupling measures the number of *incoming* connections to a code artifact (component, class, function, and so on). *Efferent* coupling measures the *outgoing* connections to other code artifacts. For virtually every platform tools exist that allow architects to analyze the coupling characteristics of code in order to assist in restructuring, migrating, or understanding a code base.

Why Such Similar Names for Coupling Metrics?

Why are two critical metrics in the architecture world that represent opposite concepts named virtually the same thing, differing in only the vowels that sound the most alike? These terms originate from Yourdon and Constantine's *Structured Design*. Borrowing concepts from mathematics, they coined the now-common afferent and efferent coupling terms, which should have been called incoming and outgoing coupling. However, because the original authors leaned toward mathematical symmetry rather than clarity, developers came up with several mnemonics to help out: *a* appears before *e* in the English alphabet, corresponding to *incoming* being before *outgoing*, or the observation that the letter *e* in efferent matches the initial letter in *exit*, corresponding to outgoing connections.

Abstractness, Instability, and Distance from the Main Sequence

While the raw value of component coupling has value to architects, several other derived metrics allow a deeper evaluation. These metrics were created by Robert Martin for a C++ book, but are widely applicable to other object-oriented languages.

Abstractness is the ratio of abstract artifacts (abstract classes, interfaces, and so on) to concrete artifacts (implementation). It represents a measure of abstractness versus implementation. For example, consider a code base with no abstractions, just a huge, single function of code (as in a single `main()` method). The flip side is a code base with too many abstractions, making it difficult for developers to understand how things wire together (for example, it takes developers a while to figure out what to do with an `AbstractSingletonProxyFactoryBean`).

The formula for abstractness appears in Equation 3-3.

Equation 3-3. Abstractness

$$A = \frac{\sum m^a}{\sum m^c}$$

In the equation, m^a represents *abstract* elements (interfaces or abstract classes) with the module, and m^c represents *concrete* elements (nonabstract classes). This metric looks for the same criteria. The easiest way to visualize this metric: consider an application with 5,000 lines of code, all in one `main()` method. The abstractness numerator is 1, while the denominator is 5,000, yielding an abstractness of almost 0. Thus, this metric measures the ratio of abstractions in your code.

Architects calculate *abstractness* by calculating the ratio of the sum of abstract artifacts to the sum of the concrete ones.

Another derived metric, *instability*, is defined as the ratio of efferent coupling to the sum of both efferent and afferent coupling, shown in Equation 3-4.

Equation 3-4. Instability

$$I = \frac{C^e}{C^e + C^a}$$

In the equation, c^e represents *efferent* (or outgoing) coupling, and c^a represents *afferent* (or incoming) coupling.

The *instability* metric determines the volatility of a code base. A code base that exhibits high degrees of instability breaks more easily when changed because of high coupling. For example, if a class calls to many other classes to delegate work, the calling class shows high susceptibility to breakage if one or more of the called methods change.

Distance from the Main Sequence

One of the few holistic metrics architects have for architectural structure is *distance from the main sequence*, a derived metric based on *instability* and *abstractness*, shown in Equation 3-5.

Equation 3-5. Distance from the main sequence

$$D = |A + I - 1|$$

In the equation, A = *abstractness* and I = *instability*.

Note that both *abstractness* and *instability* are fractions whose results will always fall between 0 and 1 (except in extreme cases of abstractness that wouldn't be practical). Thus, when graphing the relationship, we see the graph in Figure 3-2.

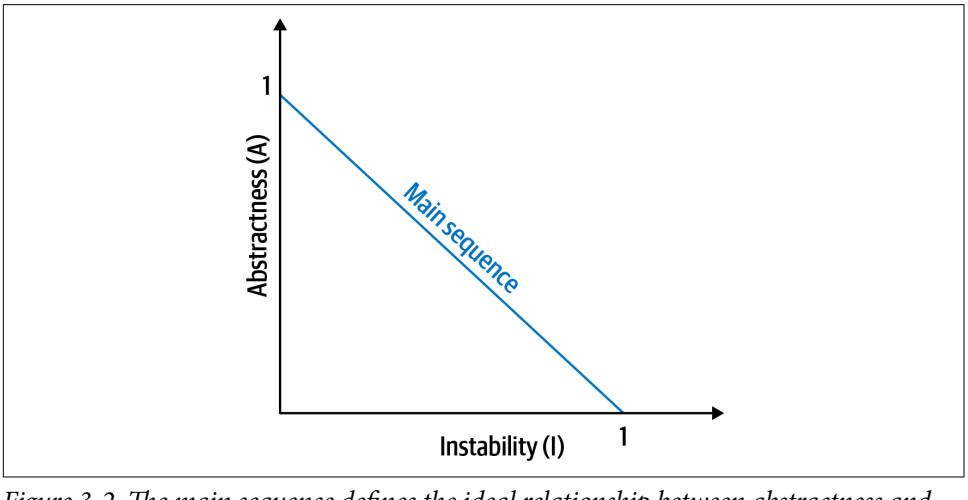

Figure 3-2. The main sequence defines the ideal relationship between abstractness and instability

The *distance* metric imagines an ideal relationship between abstractness and instability; classes that fall near this idealized line exhibit a healthy mixture of these two competing concerns. For example, graphing a particular class allows developers to calculate the *distance from the main sequence* metric, illustrated in Figure 3-3.

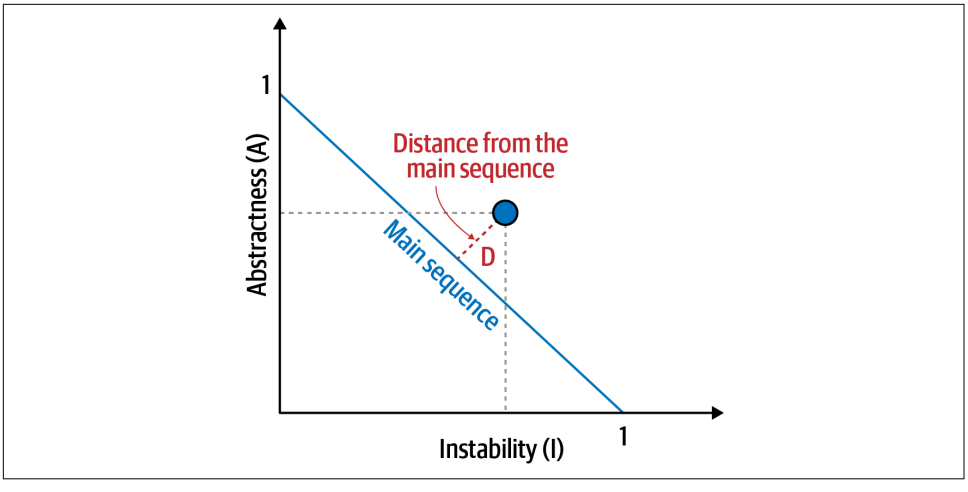

Figure 3-3. Normalized distance from the main sequence for a particular class

In Figure 3-3, developers graph the candidate class, then measure the distance from the idealized line. The closer to the line, the better balanced the class. Classes that fall too far into the upper-righthand corner enter into what architects call the *zone of uselessness*: code that is too abstract becomes difficult to use. Conversely, code that falls into the lower-lefthand corner enter the *zone of pain*: code with too much implementation and not enough abstraction becomes brittle and hard to maintain, illustrated in Figure 3-4.

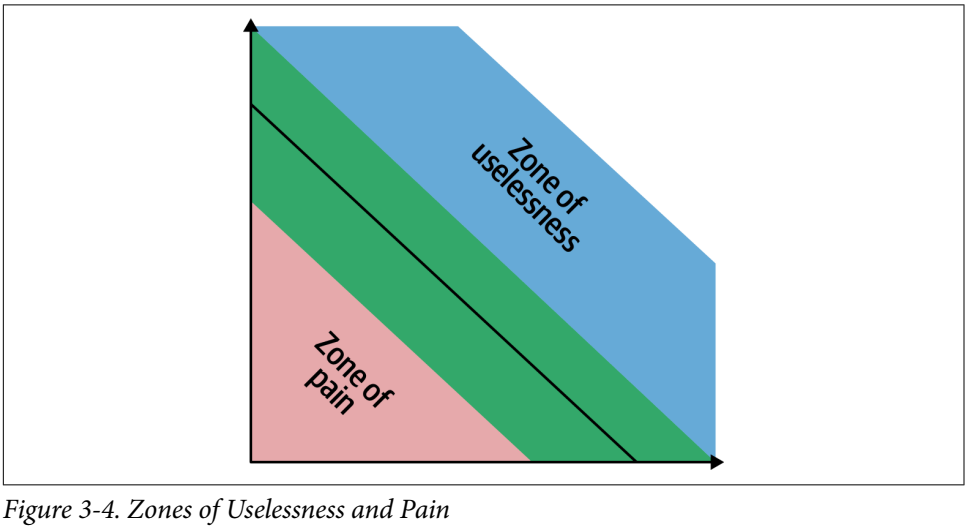

Figure 3-4. Zones of Uselessness and Pain

Tools exist in many platforms to provide these measures, which assist architects when analyzing code bases because of unfamiliarity, migration, or technical debt assessment.

Limitations of Metrics

While the industry has a few code-level metrics that provide valuable insight into code bases, our tools are extremely blunt compared to analysis tools from other engineering disciplines. Even metrics derived directly from the structure of code require interpretation. For example, cyclomatic complexity (see "Cyclomatic Complexity" on page 79) measures complexity in code bases but cannot distinguish from *essential* complexity (because the underlying problem is complex) or *accidental complexity* (the code is more complex than it should be). Virtually all code-level metrics require interpretation, but it is still useful to establish baselines for critical metrics such as cyclomatic complexity so that architects can assess which type they exhibit. We discuss setting up just such tests in "Governance and Fitness Functions" on page 82.

Notice that the previously mentioned book by Edward Yourdon and and Larry Constantine (*Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*) predates the popularity of object-oriented languages, focusing instead on structured programming constructs, such as functions (not methods). It also defined other types of coupling that we do not cover here because they have been supplanted by *connascence*.

Connascence

In 1996, Meilir Page-Jones published *What Every Programmer Should Know About Object-Oriented Design* (Dorset House), refining the afferent and efferent coupling metrics and recasting them to object-oriented languages with a concept he named *connascence*. Here's how he defined the term:

> Two components are connascent if a change in one would require the other to be modified in order to maintain the overall correctness of the system.
>
> —Meilir Page-Jones

He developed two types of connascence: *static* and *dynamic*.

Static connascence

Static connascence refers to source-code-level coupling (as opposed to execution-time coupling, covered in "Dynamic connascence" on page 50); it is a refinement of the afferent and efferent couplings defined by *Structured Design*. In other words, architects view the following types of static connascence as the *degree* to which something is coupled, either afferently or efferently:

Connascence of Name (CoN)
: Multiple components must agree on the name of an entity.

  Names of methods represents the most common way that code bases are coupled and the most desirable, especially in light of modern refactoring tools that make system-wide name changes trivial.

Connascence of Type (CoT)
: Multiple components must agree on the type of an entity.

  This type of connascence refers to the common facility in many statically typed languages to limit variables and parameters to specific types. However, this capability isn't purely a language feature—some dynamically typed languages offer selective typing, notably Clojure and Clojure Spec.

Connascence of Meaning (CoM) or Connascence of Convention (CoC)
: Multiple components must agree on the meaning of particular values.

  The most common obvious case for this type of connascence in code bases is hard-coded numbers rather than constants. For example, it is common in some languages to consider defining somewhere `int TRUE = 1; int FALSE = 0`. Imagine the problems if someone flips those values.

Connascence of Position (CoP)
: Multiple components must agree on the order of values.

  This is an issue with parameter values for method and function calls even in languages that feature static typing. For example, if a developer creates a method `void updateSeat(String name, String seatLocation)` and calls it with the values `updateSeat("14D", "Ford, N")`, the semantics aren't correct even if the types are.

Connascence of Algorithm (CoA)
: Multiple components must agree on a particular algorithm.

  A common case for this type of connascence occurs when a developer defines a security hashing algorithm that must run on both the server and client and produce identical results to authenticate the user. Obviously, this represents a high form of coupling—if either algorithm changes any details, the handshake will no longer work.

Dynamic connascence

The other type of connascence Page-Jones defined was *dynamic connascence*, which analyzes calls at runtime. The following is a description of the different types of dynamic connascence:

Connascence of Execution (CoE)
: The order of execution of multiple components is important.

  Consider this code:

```
email = new Email();
email.setRecipient("foo@example.com");
email.setSender("me@me.com");
email.send();
email.setSubject("whoops");
```

  It won't work correctly because certain properties must be set in order.

Connascence of Timing (CoT)
: The timing of the execution of multiple components is important.

  The common case for this type of connascence is a race condition caused by two threads executing at the same time, affecting the outcome of the joint operation.

Connascence of Values (CoV)
: Occurs when several values relate on one another and must change together.

  Consider the case where a developer has defined a rectangle as four points, representing the corners. To maintain the integrity of the data structure, the developer cannot randomly change one of points without considering the impact on the other points.

  The more common and problematic case involves transactions, especially in distributed systems. When an architect designs a system with separate databases, yet needs to update a single value across all of the databases, all the values must change together or not at all.

Connascence of Identity (CoI)
: Occurs when multiple components must reference the same entity.

The common example of this type of connascence involves two independent components that must share and update a common data structure, such as a distributed queue.

Architects have a harder time determining dynamic connascence because we lack tools to analyze runtime calls as effectively as we can analyze the call graph.

Connascence properties

Connascence is an analysis tool for architect and developers, and some properties of connascence help developers use it wisely. The following is a description of each of these connascence properties:

Strength

Architects determine the *strength* of connascence by the ease with which a developer can refactor that type of coupling; different types of connascence are demonstrably more desirable, as shown in Figure 3-5. Architects and developers can improve the coupling characteristics of their code base by refactoring toward better types of connascence.

Architects should prefer static connascence to dynamic because developers can determine it by simple source code analysis, and modern tools make it trivial to improve static connascence. For example, consider the case of *connascence of meaning*, which developers can improve by refactoring to *connascence of name* by creating a named constant rather than a magic value.

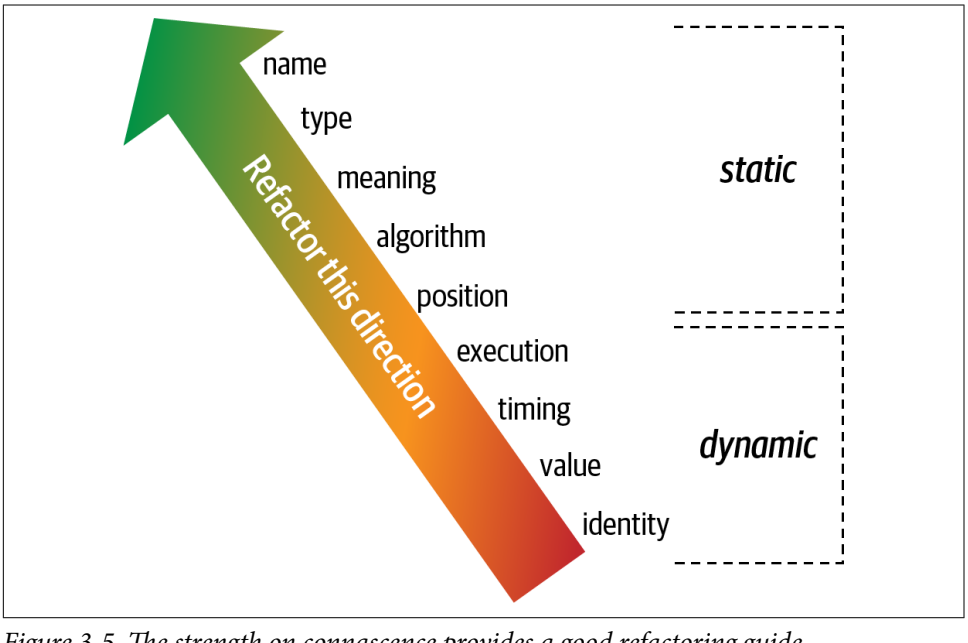

Figure 3-5. The strength on connascence provides a good refactoring guide

Locality

The *locality* of connascence measures how proximal the modules are to each other in the code base. Proximal code (in the same module) typically has more and higher forms of connascence than more separated code (in separate modules or code bases). In other words, forms of connascence that indicate poor coupling

when far apart are fine when closer together. For example, if two classes in the same component have connascence of meaning, it is less damaging to the code base than if two components have the same form of connascence.

Developers must consider strength and locality together. Stronger forms of connascence found within the same module represent less code smell than the same connascence spread apart.

Degree

The *degree* of connascence relates to the size of its impact—does it impact a few classes or many? Lesser degrees of connascence damage code bases less. In other words, having high dynamic connascence isn't terrible if you only have a few modules. However, code bases tend to grow, making a small problem correspondingly bigger.

Page-Jones offers three guidelines for using connascence to improve systems modularity:

1. Minimize overall connascence by breaking the system into encapsulated elements
2. Minimize any remaining connascence that crosses encapsulation boundaries
3. Maximize the connascence within encapsulation boundaries

The legendary software architecture innovator Jim Weirich repopularized the concept of connascence and offers two great pieces of advice:

> Rule of Degree: convert strong forms of connascence into weaker forms of connascence
>
> Rule of Locality: as the distance between software elements increases, use weaker forms of connascence

Unifying Coupling and Connascence Metrics

So far, we've discussed both coupling and connascence, measures from different eras and with different targets. However, from an architect's point of view, these two views overlap. What Page-Jones identifies as static connascence represents degrees of either incoming or outgoing coupling. Structured programming only cares about in or out, whereas connascence cares about how things are coupled together.

To help visualize the overlap in concepts, consider Figure 3-6. The structured programming coupling concepts appear on the left, while the connascence characteristics appear on the right. What structured programming called *data coupling* (method calls), connascence provides advice for how that coupling should manifest. Structured programming didn't really address the areas covered by dynamic connascence; we encapsulate that concept shortly in "Architectural Quanta and Granularity" on page 92.

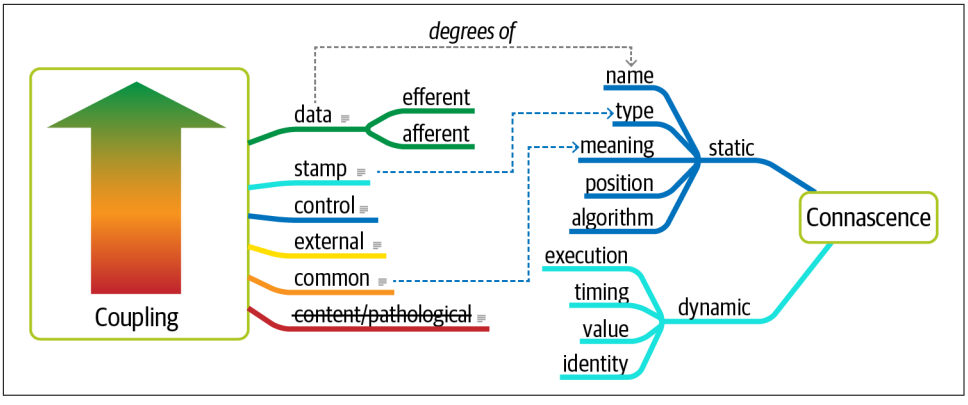

Figure 3-6. Unifying coupling and connascence

The problems with 1990s connascence

Several problems exist for architects when applying these useful metrics for analyzing and designing systems. First, these measures look at details at a low level of code, focusing on code quality and hygiene than necessarily architectural structure. Architects tend to care more about *how* modules are coupled rather than the *degree* of coupling. For example, an architect cares about synchronous versus asynchronous communication, and doesn't care so much about how that's implemented.

The second problem with connascence lies with the fact that it doesn't really address a fundamental decision that many modern architects must make—synchronous or asynchronous communication in distributed architectures like microservices? Referring back to the First Law of Software Architecture, everything is a trade-off. After we discuss the scope of architecture characteristics in Chapter 7, we'll introduce new ways to think about modern connascence.

From Modules to Components

We use the term *module* throughout as a generic name for a bundling of related code. However, most platforms support some form of *component*, one of the key building blocks for software architects. The concept and corresponding analysis of the logical or physical separation has existed since the earliest days of computer science. Yet, with all the writing and thinking about components and separation, developers and architects still struggle with achieving good outcomes.

We'll discuss deriving components from problem domains in Chapter 8, but we must first discuss another fundamental aspect of software architecture: architecture characteristics and their scope.